Starting with Nemotron-CC or Marin data: the complete pretraining design

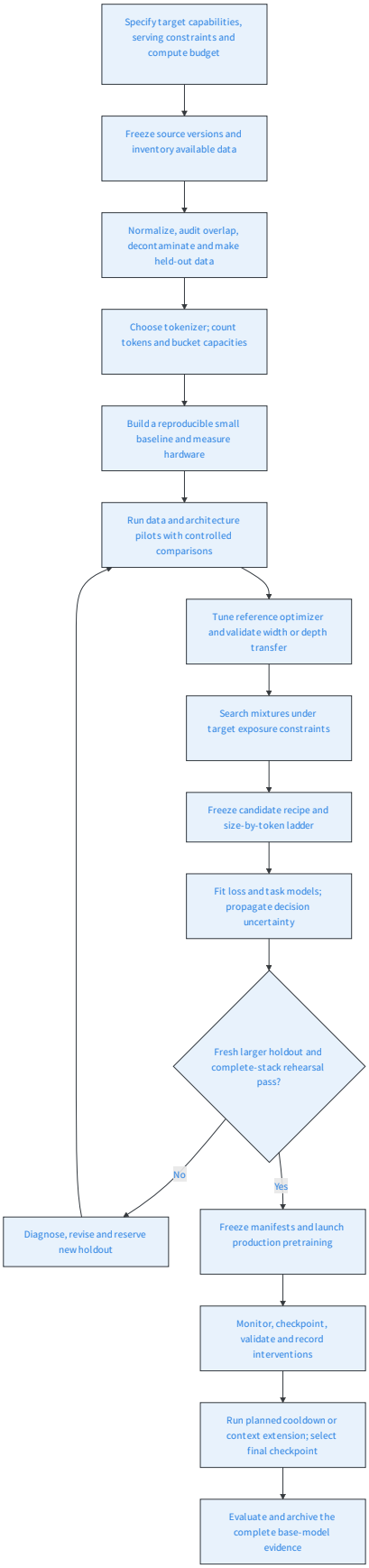
> **Note**
>
> For the latest conversation evidence and **96-GPU × 30-day** allocation, see the industry audit, data-engineering companion, trainer playbook, checkpoint evaluation, and current numerical plan. Earlier 128×40 examples remain historical alternatives.

Having a corpus is the beginning of the experiment. The next goal is a versioned, executable baseline whose data exposures, optimization, evaluations and cost are understood. Then use small experiments to choose a mixture and training recipe, validate transfer, and only then launch the target model.

This is our proposed workflow, informed by the OLMo effort, Marin effort, and Nemotron reports. It specifies decisions and gates. Exact model sizes and numerical hyperparameters require a target workload, available hardware, token budget and measured pilot throughput; the corpus name alone does not determine them.

For numerical choices under the supplied cluster budget, use the 128-H100 candidate plan: it instantiates this workflow with architecture pairs, actual data URLs/revisions, token exposures and a bounded experiment matrix.

The full process at a glance



There are two independent meanings of “held out”: **data** unseen during optimization and **runs/scales** excluded from recipe selection or fitting. You need both.

1. Decide what the final model must do

Write the decision before choosing a ladder. Is the goal a strong base model under fixed training FLOPs, low serving latency at a chosen size, better code/math performance, or an accurate prediction of a prescribed recipe? Set context length, languages, capability floors and serving-memory limits.

Choose the primary comparison metric and fixed task suite. A mixture that improves common-text loss may hurt code; a lower training loss on an easier training distribution is not automatically a better model. Make domain weights in the validation objective a deliberate product/research decision.

Artifact: a target statement and acceptance rules. Example placeholders:

```
objective: select_size_and_token_horizon_under_budget
workload: specify_languages_domains_and_context
training_budget: specify_total_accelerator_time_and_FLOPs
serving_constraints: specify_memory_latency_and_concurrency
primary_metric: fixed_weighted_validation_next_token_loss
domain_floors: specify_required_task_scores
holdout_rule: freeze_prediction_and_tolerance_before_target_runs
```

These are planning fields, not a launchable training configuration.

2. Establish exactly which data you have

If starting from Nemotron-CC: distinguish the original release from CC-v2/v2.1, CC-Math and other later variants. Record the actual snapshot, whether it includes quality annotations or synthetic material, tokenization assumptions, and what artifacts are accessible. The original report's 6.3T tokens combine 4.4T deduplicated original and 1.9T synthetic tokens; that is not the inventory of every later release. Nemotron-CC v2, Table 4.

If starting from Marin: identify a specific source catalogue, pipeline revision and mixture. Marin's processing code and upstream manifests do not mean every dataset has been rehosted as one permanent downloadable corpus. Materialize and verify required inputs before finalizing the run budget. Marin data pipeline.

For either starting point, record source identifiers, versions, provenance, document counts, byte counts, languages, annotation fields, known overlaps and any restrictions already associated with the sources. Keep raw snapshots separate from newly processed versions.

Gate: the selected data are accessible and traceable; the inventory is based on actual artifacts, not a headline release size.

3. Make training and evaluation statistically separable

Normalize records while preserving source/document lineage. Audit exact and fuzzy duplication across the full union of sources, including real text and its rewrites. Inspect candidate duplicate clusters before applying aggressive rules to code or structured text. An already processed corpus may need an overlap audit rather than a complete destructive reimplementation of its original cleaning.

Define held-out data at document or duplicate-group level rather than splitting neighboring tokens from the same document into train and validation. For each domain, preserve a fixed validation set with the same raw-text definition across candidate mixtures. Decontaminate against the declared benchmark sets using an explicit, versioned matching rule. Known exact-overlap removal does not prove paraphrase-free evaluation.

Freeze the evaluation objective before mixture search. Separate development tasks used to select a recipe from final evidence wherever possible, and disclose any task used during selection. OLMo's later integration work and Marin's decontamination pipeline are useful examples, not universally sufficient filters. OLMo 3, §§3.4–3.5; Marin pipeline.

Artifacts: normalization/dedup manifests, split IDs, benchmark-overlap audit, fixed validation distribution and evaluator version.

4. Choose tokenization and define mixture buckets

Choose a tokenizer compatible with the model family and evaluation plan. If the tokenizer itself is an experimental variable, compare candidates on identical raw text with byte-normalized metrics and full head/embedding FLOP accounting. Do not mix its change into the first optimizer comparison.

Tokenize representative samples first to catch pathological records and estimate compression. Then tokenize the frozen corpus or use an existing compatible tokenized snapshot with verified provenance. Preserve EOS/document boundaries and declare packing/masking rules.

Count post-processing unique tokens U_j for each bucket. Start with interpretable source/domain/quality buckets. Bucket count should be supportable by the mixture-search budget; a 200-bucket taxonomy is not automatically suitable for a small project.

Create a table with bucket name, source snapshot, available unique tokens, quality tier, planned sampling weight and epoch cap. Every later filtering update invalidates the old capacity count.

Gate: token counters and validation masks are correct, and no mixture silently assumes more unique data than are available.

5. Build a small executable baseline

Use one target-compatible architecture family and a documented training stack. For a first controlled study, a dense Transformer is often a simpler baseline than a new hybrid/MoE system; choose MoE if its serving/memory trade-offs are part of the objective and the infrastructure can support it. This is a project-design recommendation, not evidence that dense is universally superior.

Freeze normalization, positional encoding, activation/FFN rule, attention/GQA geometry, embedding tying, context length, initialization, parameterization and numerical precision. Use a simple corpus-proportional mixture and, where useful, a repetition-capped alternative.

Run short jobs to verify token accounting, loss behavior, evaluation, gradient clipping and checkpoint resume. Measure sustained throughput and peak memory on the intended stack. With aggregate throughput r processed tokens/second, the idealized wall time is $D_{\text{processed}}/r$; add preprocessing, evaluation, checkpointing and recovery overhead. Keep scored tokens distinct from processed tokens when reporting compute efficiency.

Gate: the baseline is reproducible, stable and affordable. Fix infrastructure errors before interpreting a scientific ablation.

6. Calibrate architecture and optimizer separately

First choose the transfer question. To test μP width transfer, hold depth, batch, horizon and data fixed while changing width under one complete parameterization. A base LR can transfer even though actual hidden-matrix LRs change with width. Confirm this behavior at at least another width rather than assuming a library flag guarantees it.

To choose depth, compare matched-budget width/depth alternatives separately and retune sufficiently. AI2's 16-layer small models followed by 32/40-layer targets are a practiced prediction recipe, not proof that depth changes require no recalibration. AI2 Table 1; width/depth evidence and decision flow.

At a reference configuration, sweep LR and batch, then refine weight decay and any important residual/initialization settings. Give candidate optimizers comparable tuning effort. Compare the declared annealed endpoint, not just an arbitrary stable-phase checkpoint.

If using an empirical transfer law, record its data, parameterization and units. Power Lines' batch coefficients use sequences of 2,048 tokens, while Step Law's reported batch law uses tokens. The former also finds a changing optimal decay timescale across TPP. Power Lines v2; optimizer-timescale companion.

Artifact: a complete provisional recipe and measured transfer range, including parameter-group hyperparameters and precision.

7. Optimize data for the intended horizon

With total training tokens D and bucket fractions w_j, compute $e_j = Dw_j/U_j$. An illustrative 100B-token run that assigns 10% to a 1B-token bucket repeats it about ten times. A three-exposure cap limits its weight to 3%. This arithmetic should happen before the training run.

Compare a few source/quality ablations against a fixed baseline. If budget allows a mixture swarm, train proxies on sampled feasible weights, fit per-task response models, reserve unseen mixtures for validation, and optimize under the capacity/epoch constraints. Preserve target exposure in the proxy: if proxy tokens are scaled by s, scale bucket pools by s too when repetition is part of the question.

OLMo 3's reported 30M/3B proxies are a concrete starting reference; Marin uses exposure-matched proxies in its later mixture effort. Neither makes 30M or a particular swarm size universally sufficient. Confirm the selected mixture at an intermediate scale and inspect domain regressions. OLMo 3 §3.4.4; Marin data pipeline.

For scarce late-stage sources, a common-checkpoint microanneal can be more informative than retraining from scratch. OLMo 3's standard comparison uses 5B candidate + 5B web versus 10B web. It measures late-stage marginal value, so integrate promising sources in a longer complete mixture before extrapolating to the whole run. OLMo 3 §3.5.1.

Gate: the mixture beats declared baselines on fixed validation data and transfers beyond the smallest proxy.

8. Design the size/token ladder

After the cheap pilots, freeze a candidate family and tuning policy. Decide whether the ladder forecasts a prescribed recipe or estimates a tuned frontier. Vary both size and horizon when the decision involves allocating parameters versus tokens.

An illustrative layout is:

Role	Parameter tiers	Horizons	Purpose
Fit	$N_0, 2N_0, 4N_0$	$D/N \in \{q_*/2, q_*, 2q_*\}$	Nine size/token points around intended TPP q_*
Development	An intermediate larger tier	Selected horizons	Choose fit form and diagnose transfer
Final holdout	A larger unused tier	Target-like horizon plus a useful contrast	Test the frozen prediction

Choose N_0, tier spacing and horizons from the actual budget; this table is a design template, not a prescribed affordable run count. Add independent seeds where the pilot shows noise could change the decision. If allocation is the main question, use multiple IsoFLOP slices instead or alongside the Cartesian grid.

Reserve tuning, replication, failures, annealing branches and final holdouts in the compute ledger. Do not assign all resources to fitting points. Count shared trunks once. Keep the final-holdout outcomes unavailable during recipe selection.

Artifact: a run matrix with full configurations, budgets, parent checkpoints, seed roles and fit/development/test labels.

9. Fit and decide using uncertainty

Fit a declared loss model on comparable endpoints. An additive surface is a useful baseline; compare justified alternatives on development scales, inspect two-dimensional residuals and repeat optimization from multiple starts. Multiple starts do not guarantee global optimality.

Bootstrap independent runs or shared-prefix clusters, refit, and recompute the target decision for each replicate. Report a distribution over candidate margins or feasible allocations, not only intervals on coefficients. Include task forecasts only when their intermediate metrics and calibration have passed independent checks. Allocation derivation; inference companion.

Select among feasible architecture/token choices using quality, memory, throughput and lifetime serving constraints. Recheck the winning region directly; if several choices are within uncertainty, practical cost can decide among them.

Gate: the forecast is useful for the decision, with residuals and intervals that support its claimed precision.

10. Validate the whole stack before the large run

Combine the chosen data, architecture, optimizer, schedule, precision and checkpoint-selection policy in one candidate. Individual improvements need not add. Test a fresh larger tier and an intermediate production-stack rehearsal at target-like TPP and data exposure.

Check long-horizon gradients/update ratios, residual norms, nonfinite values, loss spikes and, for MoE, routing imbalance/drop behavior. Verify optimizer-state and data-sampler resume. If changing research to production kernels or distributed strategy, compare updates and short trajectories under explicit numerical tolerances.

Ultra's report is a practical warning from real training: an output-gradient accumulation precision change contributed to an early divergence; a later failure required an annealing intervention. This is evidence for a production-like rehearsal and instrumentation, not a universal FP32/BF16 rule. Nemotron Ultra §2.7.

Once a holdout result changes the recipe, classify it as development evidence and reserve a new test.

11. Launch and monitor production pretraining

Freeze code/data/tokenizer versions, complete configuration, evaluator, seeds, hardware/parallelism settings, budget and intervention policy. Begin with the validated baseline update, not a new untested optimization introduced at scale.

Monitor fixed-distribution and per-domain validation loss, task metrics, throughput, memory and numerical health. Schedule evaluations at comparable progress fractions where useful. Save model, optimizer, scheduler, RNG and data-position state; verify that recovery preserves sampling and step/token counts.

If a run deviates, distinguish corrupted data, a broken update, numerical instability, infrastructure failure and genuine recipe mismatch. Preserve the original configuration and log interventions. A rescued run remains valuable engineering evidence, but do not silently treat it as an unchanged fixed-recipe validation point.

12. Finish the model you actually intend to deliver

If the plan includes quality-focused cooldown, checkpoint averaging or context extension, validate and execute those stages using the declared mixtures and endpoint policy. Keep the data-mixture transition separate from the start of LR decay: Nemotron Nano's report places them at different fractions of training. Nano §§2.3–2.5.

Evaluate the final base checkpoint on fixed loss and task suites, including domain floors and effective context length. If delivery includes post-training, run the planned standardized probe or full post-training pipeline and report its contribution separately.

Archive the source manifests, actual token exposures, run lineage, failed attempts, chosen checkpoint policy, fit predictions, holdout errors, final metrics and runtime costs. Release or retain the artifacts appropriate to the project. A model checkpoint without its data and optimization lineage cannot substantiate the scientific claims of the ladder.

What I would do first in this repository

1. Choose one concrete CC or Marin snapshot and produce its actual bucket/capacity manifest.
2. Define the target workload, fixed evaluation set and available accelerator budget.
3. Build one small target-compatible baseline and measure throughput, loss variance and resume behavior.
4. Use those measurements to fill the reference sweep and ladder matrix; then run controlled data/shape pilots.

Those four artifacts let us choose numerical sizes and token budgets responsibly. Until they exist, copying a published architecture or a TPP rule would conceal the most important unknowns.

Blog reading guide · Source housekeeping · OLMo deep dive · Marin deep dive